

RMTPP

BY JOSH JOB JOSEPH

PACKAGE USED: PYTORCH

- PyTorch is an open-source deep learning framework developed by Meta (Facebook).
- Provides tools for building and training neural networks with GPU acceleration.
- Known for its dynamic computation graph (define-by-run), making it flexible for research.

PACKAGE USED: PYTORCH

Why PyTorch for RMTTP?

1. Automatic Differentiation

- PyTorch's autograd computes gradients automatically (e.g., for backpropagation in `self.W_y`, `self.W_h`).
- Critical for training the intensity function and marker prediction layers.

2. GPU Acceleration

- Enables fast processing of sequences (e.g., markers and times tensors) via CUDA.
- Used in this code via `device = markers.device` to leverage GPU if available.

3. Dynamic Computation Graphs

- RMTTP's recurrent structure (hidden state updates in `forward_sequence`) benefits from PyTorch's dynamic graphs.
- Easier debugging compared to static graph frameworks.

4. Rich NN Module Library

- Pre-built layers (e.g., `nn.Embedding`, `nn.Linear`) simplify model architecture.
- Loss functions like `F.cross_entropy` are optimized for stability.

5. Numerical Integration Support

- PyTorch's tensor operations enable efficient intensity calculations (e.g., `torch.cumsum` in `predict_next_time`).

PACKAGE USED: PYTORCH

Feature	Use Case in RMTTP
<code>nn.Embedding</code>	Embed event markers (<code>self.embed</code>)
<code>nn.Linear</code>	Hidden state updates (<code>self.W_y</code> , <code>self.W_h</code>)
<code>autograd</code>	Gradient computation for NLL loss
<code>torch.relu</code>	Nonlinear hidden state activation
<code>torch.linspace</code>	Time grid for intensity evaluation

<pre> class RMTPP(nn.Module): def __init__(self, num_markers, embed_dim, hidden_dim): super().__init__() self.num_markers = num_markers self.embed_dim = embed_dim self.hidden_dim = hidden_dim # Marker embedding layer self.embed = nn.Embedding(num_markers, embed_dim) self.embed_bias = nn.Parameter(torch.zeros(embed_dim)) # Hidden state update weights self.W_y = nn.Linear(embed_dim, hidden_dim, bias=False) self.W_t = nn.Linear(1, hidden_dim, bias=False) self.W_h = nn.Linear(hidden_dim, hidden_dim, bias=False) self.b_h = nn.Parameter(torch.zeros(hidden_dim)) # Marker generation weights self.gen_marker = nn.Linear(hidden_dim, num_markers) # Intensity function parameters self.v_t = nn.Linear(hidden_dim, 1, bias=False) self.w_t = nn.Parameter(torch.ones(1)) self.b_t = nn.Parameter(torch.zeros(1)) </pre>	Paper Concept	Code Implementation	Purpose
	Event embedding	self.embed = nn.Embedding(...)	Maps discrete event markers (e.g., "click", "purchase") to dense vectors
	History encoding	self.W_y, self.W_h, self.W_t	Combines current event, time delay, and past hidden state
	Intensity function	self.v_t, self.w_t, self.b_t	Models $\lambda(t) = \exp(v^T h_j + w(t-t_j) + b)$
	Marker prediction	self.gen_marker = nn.Linear(...)	Predicts next event type from hidden state


```
def forward_event(self, y_j, d_j, h_prev):
    embedded = self.embed(y_j) + self.embed_bias
    d_j = d_j.unsqueeze(1)
    h_j = torch.relu(
        self.W_y(embedded) +
        self.W_t(d_j) +
        self.W_h(h_prev) +
        self.b_h
    )
    return h_j
```

HIDDEN LAYER

$$h_j = \max \left\{ W^y y_j + W^t t_j + W^h h_{j-1} + b_h, 0 \right\}. \tag{9}$$

t_j (e.g., like the inter-event duration d_j = t_j - t_{j-1})

Code	Mathematical Representation	Purpose
embedded = self.embed(y_j) + self.embed_bias	$y_j \in \mathbb{R}^d$	Embeds event marker into continuous space
d_j = d_j.unsqueeze(1)	$\Delta t_j \in \mathbb{R}$	Prepares time delta for linear transformation
self.W_y(embedded)	$W_y y_j$	Projects embedded event to hidden space
self.W_t(d_j)	$W_t \Delta t_j$	Encodes time delay influence (unique to RMTTPP)
self.W_h(h_prev)	$*W_h h_{\{j-1\}}^*$	Carries forward historical context
torch.relu(...)	$ReLU(x)=(x)+=\max(0,x)$	Nonlinear activation (ReLU)

MARKER GENERATION

```
# Marker generation weights
self.gen_marker = nn.Linear(hidden_dim, num_markers)
```

Matches: V^y (weights) and b^y (bias)

PyTorch `nn.Linear` automatically includes a bias term unless `bias=False`.

Dimensions: $(\text{hidden_dim} \rightarrow \text{num_markers}) = (d \times K)$ (transposed compared to paper)

CLASS `torch.nn.Linear(in_features, out_features, bias=True, device=None, dtype=None)` [\[SOURCE\]](#)

Applies an affine linear transformation to the incoming data: $y = xA^T + b$.

```
def predict_next_marker(self, h_j):
    logits = self.gen_marker(h_j)
    return F.softmax(logits, dim=-1)
```

Linear transform: $V^y h_j + b^y$ (via `self.gen_marker(h_j)`)

Softmax: Normalizes to probabilities.

`torch.nn.functional.softmax(input, dim=None, _stacklevel=3, dtype=None)` [\[SOURCE\]](#)

Apply a softmax function.

Softmax is defined as:

$$\text{Softmax}(x_i) = \frac{\exp(x_i)}{\sum_j \exp(x_j)}$$

It is applied to all slices along `dim`, and will re-scale them so that the elements lie in the range $[0, 1]$ and sum to 1.

Marker Generation. Given the learned representation h_j , we model the marker generation with a multinomial distribution by

$$P(y_{j+1} = k | h_j) = \frac{\exp(V_{k,:}^y h_j + b_k^y)}{\sum_{k=1}^K \exp(V_{k,:}^y h_j + b_k^y)}, \quad (10)$$

where K is the number of markers, and $V_{k,:}^y$ is the k -th row of matrix V^y .

CONDITIONAL INTENSITY

```
def conditional_intensity(self, h_j, t_j, t):
    past_influence = self.v_t(h_j).squeeze(-1)

    if t.dim() > 1:
        delta_t = t - t_j.unsqueeze(-1)
    else:
        delta_t = t - t_j

    mask = (delta_t >= 0).float()
    current_influence = self.w_t * delta_t
    intensity = torch.exp(
        past_influence.unsqueeze(-1) +
        current_influence +
        self.b_t
    )
    return intensity * mask
```

Conditional Intensity. Based on h_j , we can now formulate the conditional intensity function by

$$\lambda^*(t) = \exp \left(\underbrace{v^{t\top} \cdot h_j}_{\text{past influence}} + \underbrace{w^t(t - t_j)}_{\text{current influence}} + \underbrace{b^t}_{\text{base intensity}} \right), \quad (11)$$

Term in Paper (Eq. 11)	Code Implementation	Purpose
$v^T h_j$ (Past influence)	<code>self.v_t(h_j).squeeze(-1)</code>	Projects hidden state to scalar via learned weights <code>v_t</code>
$w_t(t-t_j)$ (Current influence)	<code>self.w_t * (t - t_j)</code>	Learns time scaling (growth/decay) with parameter <code>w_t</code>
b_t (Base intensity)	<code>self.b_t</code>	Baseline event rate (trainable scalar)
exp(·)	<code>torch.exp(past + current + b_t)</code>	Ensures $\lambda^*(t) > 0$
Causality Mask	<code>(delta_t >= 0).float()</code>	Sets $\lambda^*(t)=0$ for $t < t_j$ (enforces temporal causality)


```
def predict_next_time(self, h_j, t_j, T=10.0, steps=1000):
    device = h_j.device
    time_grid = torch.linspace(0, T, steps=steps, device=device)
    t_eval = t_j + time_grid

    intensity = self.conditional_intensity(
        h_j,
        torch.tensor([t_j], device=device),
        t_eval.unsqueeze(0)
    ).squeeze()

    # Trapezoidal integration for cumulative intensity
    dt = T / (steps - 1)
    integral = torch.cat([
        torch.zeros(1, device=device),
        torch.cumsum(0.5 * (intensity[:-1] + intensity[1:]) * dt, dim=0)
    ])

    survival = torch.exp(-integral)
    density = intensity * survival

    # Expected time calculation
    t_values = t_eval
    expectation = torch.trapz(t_values * density, t_values)
    return expectation.item()
```

Term	Code Implementation	Math Equivalent
$\lambda^*(t)$	<code>conditional_intensity(h_j, t_j, t_eval)</code>	$\exp(v^T h_j + w_t \Delta t + b_t)$
$\int \lambda^*(t) dt$	Trapezoidal rule on <code>intensity</code>	$\sum_i \frac{\lambda_i + \lambda_{i+1}}{2} \Delta t$
$S(t)$	<code>torch.exp(-integral)</code>	$\exp(-\int \lambda^*(t) dt)$

Step	Code Snippet	Explanation
1. Time Grid Setup	<code>time_grid = torch.linspace(0, T, steps=steps)</code>	Creates steps points between 0 and T (e.g., 1000 steps over 10 sec).
2. Shift to t _j	<code>t_eval = t_j + time_grid</code>	Evaluates $\lambda \star (t)$ starting from t _j .
3. Intensity Calculation	<code>intensity = self.conditional_intensity(...).squeeze()</code>	Computes $\lambda \star (t)$ for all t_eval (uses Eq. 11).
4. Cumulative Intensity	<code>integral = torch.cumsum(0.5 * (intensity[:-1] + intensity[1:]) * dt</code>	Approximates $\int \lambda \star (t) dt$ via trapezoidal rule.
5. Survival Probability	<code>survival = torch.exp(-integral)</code>	$S(t) = \exp(-\int \lambda \star (t) dt)$ (probability of no event by t).
6. Event Density	<code>density = intensity * survival</code>	$\lambda \star (t) S(t)$: Instantaneous probability density of next event.
7. Expected Time	<code>expectation = torch.trapz(t_values * density, t_values)</code>	$E[t_{j+1}] = \int t \cdot \lambda \star (t) S(t) dt$

TIME GENERATION

RMTPP LOSS COMPUTATION

```
def compute_loss(self, markers, times):
    batch, seq_len = markers.shape
    hidden_states, durations = self.forward_sequence(markers, times)
    total_marker_nll = 0
    total_time_nll = 0
    num_events = seq_len - 1

    if num_events < 1:
        return 0, 0, 0

    for j in range(num_events):
        logits = self.gen_marker(hidden_states[j])

        marker_nll = F.cross_entropy(logits,
                                     markers[:, j+1],
                                     reduction='sum')

        total_marker_nll += marker_nll

        d_next = durations[:, j+1]
        A = self.v_t(hidden_states[j]).squeeze(1) + self.b_t
        s = self.w_t * d_next
        exp_A = torch.exp(A)

        if torch.abs(self.w_t) < 1e-10:
            integral = exp_A * d_next
        else:
            integral = exp_A * (torch.exp(s) - 1) / self.w_t

        log_f = A + s - integral
        time_nll = (-log_f).sum()
        total_time_nll += time_nll

    return total_marker_nll, total_time_nll, num_events * batch
```

Component	Code Snippet	Mathematical Equivalent		
Marker NLL	F.cross_entropy(logits, markers[:, j+1])	-logP(yj+1 // hj)		
Time NLL	-log_f.sum()	-logλ(tj+1)+∫tjtj+1λ(t)dt		
Component	Code Implementation	Mathematical Equivalent	Purpose	
Past Influence (A)	A = self.v_t(h_j).squeeze(1) + self.b_t	A=vThj+bt	Encodes how history affects current intensity.	
Time Scaling (s)	s = self.w_t * d_next	s=wtΔtjs	Captures linear time dependence in intensity.	
Integral of λ(t)	integral = exp_A * (torch.exp(s) - 1) / w_t	∫tj+1λ(t)dt	Closed-form solution for cumulative intensity (avoids numerical integration).	
Log-Likelihood	log_f = A + s - integral	logλ(tj+1)-∫λ(t)dt	Log of the temporal event likelihood	
Time NLL	time_nll = (-log_f).sum()	-logLtime	Penalizes incorrect time predictions; minimized during training.	